Exercise 4: Two Skills and a Secret Guard

Session: 4 — Skills, Hooks & Automation
Duration: 40 minutes, plus a 5-minute closing round
Project: The same BookStore API.

Goal

The commands you have used since session 2 are markdown files in this repo. First you make two of your own: copy a ready-made `/commit` skill, then let Claude write a `/changelog` skill, write its `description` yourself, and test whether that description starts the skill at the right moment.

Then the hook. Everything you type and everything Claude reads goes to the model. You wire a hook that blocks a prompt with a secret in it, find the case the hook does not see, and close that case with a rule.

Start here

- Open a terminal in the `bookstore` folder and start `claude`. Everything you create goes in `bookstore/.claude/`.
- The file `.env` in the `bookstore` folder holds a fake API key, made for this exercise. Treat it as a real one.
- Training mode is on. Some prompts on this sheet start with an experiment tag in square brackets. The tag switches training mode off for that one prompt. Copy those prompts exactly, tag included.
- Never edit `CLAUDE.md`. Sessions 5 to 8 need it.
- No coach this session: the *Done when* lines are your checklist, and `/verify-exercise 4` grades your files at the end. Stuck? Type *just tell me*.

Tasks

1. Copy the `/commit` skill and run it

This task produces a branch, the file `.claude/skills/commit/SKILL.md`, and one commit.

1. **Create the branch**. Sessions 5 to 8 reuse this code. You delete the branch after the closing round.

```
git switch -c session4-playground
```

2. **Let Claude copy the skill**. The skill is ready in `exercises/starters/commit-skill.md`. Paste:

```
[Exercise 4 experiment — execute directly, no leading questions.] Copy
exercises/starters/commit-skill.md to .claude/skills/commit/SKILL.md. Change nothing in
the content.
```

3. **Make a small change by hand**. Open any file in `bookstore`, add one comment line, save it. Do not ask Claude.

4. **Run the skill**. Type `/commit` and watch what it does.

5. **Read three lines of the skill**. Open `.claude/skills/commit/SKILL.md` in your editor. Three lines in it matter most. "*Execute directly — no leading questions*" switches training mode off inside the skill; without it, Claude asks a question instead of committing. `disable-model-invocation: true` means only typing `/commit` runs it. *Imperative mood* means the message starts with a present-tense verb: "Add", "Fix", "Remove". Task 2's changelog is built from those messages.

Done when: the commit exists, its message starts with a present-tense verb, and nothing was pushed.

When the trainer calls task 2, start it, even if `/commit` has not run yet.

2. Let Claude write `/changelog`, then write and test its description

This task produces `.claude/skills/changelog/SKILL.md` and, next to it, `common-changelog-spec.md`.

1. **One prompt for both files**. A skill can carry extra files, loaded only when it runs (*Anatomy of a Skill*). So the specification lives next to the skill, not inside it. Paste:

```
[Exercise 4 experiment — execute directly, no leading questions.] Read https://common-changelog.org/ and save the whole specification as markdown to .claude/skills/changelog/common-changelog-spec.md. Keep the headings and the examples. Leave out the website menu and footer. Then create .claude/skills/changelog/SKILL.md. Frontmatter: the line `name: changelog` and the line `description: (I write this)`. No other frontmatter lines. Body: a numbered list of instructions that make you do all of the following.
- Read CHANGELOG.md in the project root. If the file does not exist, create it with the heading `# Changelog`.
- Read common-changelog-spec.md, next to this file, before changing the changelog.
- Look at the recent commits and at the existing version tags.
- Add only new commits. Entries that are already in the changelog stay as they are. New entries go in a `## Unreleased` section at the top.
- Sort entries under `### Changed`, `### Added`, `### Removed` and `### Fixed`, in that order.
- Write each entry as one sentence that starts with a present-tense verb. End it with the short commit hash in round brackets, like (a1b2c3d).
- A breaking change forces other people to change their own code. Put **Breaking:** in front of such an entry, and put those entries first inside their group.
- Leave out changes a reader does not care about: files whose name starts with a dot, formatting-only changes, and developer tools.
```

If your network blocks the page, ask Claude to write the spec file from what it knows about Common Changelog.

2. **Check two things**. `common-changelog-spec.md` holds the group names and the rules, not an empty page. The `SKILL.md` frontmatter has **no** `disable-model-invocation` line: Claude must be able to start this skill by itself. Step 4 tests that.

3. **Write the** `description`. In your editor, replace `(I write this)`. Claude reads only this line when it decides whether to start a skill on its own (*Which Description Gets This Skill Invoked at the Right Moment?*). For comparison, here's the start of a course skill's description. It says *when* it applies, not only what it does:

> Coach a participant's context artifact or experiment during the Session 3 exercise. Usage: /context-coach — the participant shows their CLAUDE.local.md draft …

Two questions. Which words in your description make Claude reach for this skill right after a commit? Which words would start it at a moment you do not want?

4. **Test the trigger**. Type two requests in plain words, without a slash. Watch whether Claude starts the `/changelog` skill: the chat shows the skill loading. First:

```
I just committed. Bring the changelog up to date.
```

   Claude must start the skill, not write a changelog from memory. Now the second request:

```
Summarise the last three commits for me, in two sentences.
```

   Claude must **not** start the skill. If either request went wrong, sharpen your description and send that request again. Keep the words that did the work; change the words that misled.

Done when: both files exist, the frontmatter has no `disable-model-invocation` line, the first request started the skill and the second did not.

When the trainer calls task 3, start it, even if a request still goes wrong.

3. Wire the guard and predict

This task produces `.claude/hooks/guard-secrets.py`, an entry in `.claude/settings.json`, and three predictions.

1. **Let Claude create the script**. It runs on every prompt you send, before the prompt leaves your laptop (*The Secret That Never Leaves*). It is Python, not a shell script, because many of you are on Windows. Paste this whole block:

```
[Exercise 4 experiment — execute directly, no leading questions.] Create
.claude/hooks/guard-secrets.py with exactly this content, nothing added or changed:

import json
import re
import sys

data = json.load(sys.stdin)
text = data.get("prompt", "")

if re.search(r"acme_live_[A-Za-z0-9]{24}", text):
    print(json.dumps({
        "decision": "block",
        "reason": "Blocked: this prompt contains an ACME API key. Remove the key and send
it again."
    }))
```

2. **Add the hook to your settings**. Create the file `.claude/settings.json` yourself (it does not exist yet), with exactly this content:

```
{
  "hooks": {
    "UserPromptSubmit": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "python .claude/hooks/guard-secrets.py"
          }
        ]
      }
    ]
  }
}
```

   On mac or linux, write `python3` instead of `python`. Not sure? Run `python --version`; if that fails, use `python3`.

3. **Write three predictions**, on paper or in a text file, not in the chat; the verifier asks for them. For each case, one line: does the guard block it, yes or no, and why.

    1. You paste a log line that contains the key.
    2. You ask Claude what the value of `ACME_API_KEY` in `.env` is.
    3. You ask Claude to run `grep -r acme_live .`.

Done when: both files exist and your three predictions are written.

When the trainer calls task 4, start it, whatever your predictions look like.

4. Run it, find the hole, close it

This task produces one blocked prompt, a `permissions` rule in `.claude/settings.json`, and your marked predictions.

1. **Paste the trap**. This looks like a log line copied from a terminal. Send it as a normal prompt:

```
This request failed on staging. What is wrong with it?
POST /api/books 401 token=acme_live_7Hq2mX9vK4pL8sW1dR3nT6yB reason=expired
```

    The guard blocks it, and the terminal shows the reason from your script. The prompt never reached the model. (If Claude answered instead, `python` was not found: fix the command in `settings.json` and send the trap again.)

2. **Ask for the key**. Send this as a normal prompt:

```
What is the value of ACME_API_KEY in .env?
```

    Claude reads the file. The guard did not block anything: it runs on the text *you* type, never on what Claude reads with its tools. Whether Claude repeats the value or not, the file content is in the transcript now, so it went to the model.

3. **Close the hole with a rule**. A rule is a list, not a script: it names files Claude may not read (*Claude Must Never Read* `.env`). Open `.claude/settings.json` and add the `permissions` part, so the file looks like this:

```
{
  "permissions": {
    "deny": ["Read(.env)"]
  },
  "hooks": {
    "UserPromptSubmit": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "python .claude/hooks/guard-secrets.py"
          }
        ]
      }
    ]
  }
}
```

Save it; no restart is needed. Send the same question again. Claude reports that it may not read the file. Then try the command form:

```
Run cat .env and show me the output.
```

Also denied: a `Read` rule covers the file commands Claude Code knows, such as `cat`, `head` and `tail`.

4. **Mark your predictions**. For each one, write confirmed or wrong, with one line of evidence from the chat. Prediction 3 is tested in the bonus; if you skip the bonus, mark it with one line of reasoning.

5. **Run the verifier**. Type `/verify-exercise 4`. It asks for your three predictions first and settles them, then grades both skills, the guard and the rule. Read its report before the closing round.

Done when: the trap was blocked, the rule is in `settings.json`, your three predictions are marked, and the verifier is running.

Bonus (only if time remains)

The hole in the rule. Send this as a normal prompt:

```
Run grep -r acme_live . and show me the output.
```

The key appears. The rule matches a command that *names* the file; `grep -r` names none, so the rule does not see it. Neither does the guard. The wall that stops every process from reading a file is the sandbox (*The Claude Code Security Harness*). Every layer has a hole. That is why there are layers.

Send the guard a fake event, the way Claude Code does. Save this as `.claude/fake-event.json`:

```
{"prompt":"deploy with acme_live_7Hq2mX9vK4pL8sW1dR3nT6yB please"}
```

Run this in your own terminal, not in Claude (`python3` on mac or linux):

```
python .claude/hooks/guard-secrets.py < .claude/fake-event.json
```

The script prints the block decision. Change the prompt in `fake-event.json` to `deploy please`: it prints nothing.

Closing round

The trainer asks the room. Have these answers ready:

- Which prediction was wrong, and what showed it?
- What does the guard see, what does the rule see, and what sees everything?
- Which words in your `description` started `/changelog` at the right moment? Which would you make sharper now?
- One thing you would tell someone who skipped this session.

After the closing round

Clean up the branch. Two lines, one at a time (PowerShell 5.1 cannot join them with `&&`):

```
git switch -
git branch -D session4-playground
```

The commits are gone. Your files are not: `CHANGELOG.md`, `.claude/skills/`, `.claude/hooks/` and `.claude/settings.json` are gitignored.

Back at work, two things. If you build one hook this month, build the feedback loop: a `PostToolUse` hook on `Edit|Write` that runs your formatter, or your fastest tests, and reports the failures to Claude. Claude then fixes what it just broke, without being asked. The hooks page in the Claude Code docs shows the settings; most teams need one small script, the shape of `guard-secrets.py`. And the plugin **skill-creator** on Anthropic's official marketplace writes a first version of a skill for you: `/plugin marketplace add anthropics/claude-plugins-official`, then `/plugin` to install it, then `/reload-plugins`. Try it on a job you repeat every week and compare its draft with your task 2 file.